

integrantes :Geovanna freitas, Giovanna souza, Gustavo ramos e Raul Duarte

## Entrega 1: Modelagem e Arquitetura - Sistema de Gestão para Doceria

### Introdução

Este documento apresenta a primeira etapa do desenvolvimento de um **Sistema de Gestão Comercial (SGC)**, focado especificamente em uma **Loja de Doces e Confeitaria**. Nosso objetivo é criar uma ferramenta que ajude o dono da doceria a organizar as vendas, controlar o estoque de doces e gerenciar seus clientes de forma mais eficiente. Este trabalho segue as diretrizes da disciplina de Arquitetura e Design de Software, buscando aplicar conceitos de modelagem e arquitetura de sistemas de forma prática.

---

### 1. Descrição do Sistema: SGC para Doceria

O **Sistema de Gestão Comercial (SGC) para Doceria** será uma aplicação desenvolvida para otimizar as operações diárias de uma loja de doces e confeitaria. Ele permitirá que o proprietário e seus funcionários realizem tarefas essenciais como registrar vendas de bolos, brigadeiros e outros doces, controlar a quantidade de produtos disponíveis no estoque e manter um cadastro dos clientes.

O sistema será dividido em duas partes principais: um **Backend** (a parte que “pensa” e guarda as informações) construído em Java com Spring Boot, que vai oferecer uma **API REST** (um conjunto de “portas” para que outras aplicações se comuniquem com ele). Essa API será segura, com autenticação por **Token (JWT)**. A outra parte será o **Frontend** (a interface que o usuário vê e interage), que pode ser uma aplicação de desktop (Swing) ou uma aplicação web, e que vai usar a API do Backend para funcionar. Todos os dados, como os doces vendidos e os clientes, serão guardados em um banco de dados **MySQL**.

---

### 2. Requisitos Funcionais (RF)

Os requisitos funcionais descrevem o que o sistema precisa fazer para ajudar a doceria. São as ações e funcionalidades que o usuário poderá realizar:

| ID   | Requisito          | Descrição                                                                                             |
|------|--------------------|-------------------------------------------------------------------------------------------------------|
| RF01 | Gestão de Clientes | O sistema deve permitir cadastrar novos clientes, ver os clientes já existentes, mudar as informações |

|             |                                     |                                                                                                                                                                                                                                                                          |
|-------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                     | deles e, se necessário, remover um cliente do cadastro.                                                                                                                                                                                                                  |
| <b>RF02</b> | <b>Validação de Cliente</b>         | Ao cadastrar um cliente, o sistema deve verificar se o CPF já existe para evitar duplicidade e se o e-mail digitado é válido.                                                                                                                                            |
| <b>RF03</b> | <b>Gestão de Produtos (Doces)</b>   | O sistema deve permitir cadastrar os doces que a loja vende (ex: bolo de chocolate, brigadeiro, cupcake), informando o nome, descrição, preço e a quantidade disponível em estoque.                                                                                      |
| <b>RF04</b> | <b>Controle de Estoque de Doces</b> | Antes de vender um doce, o sistema deve verificar se há quantidade suficiente no estoque. Se não houver, a venda não deve ser permitida. Além disso, deve ser possível definir um estoque mínimo para cada doce e o sistema pode alertar quando o estoque estiver baixo. |
| <b>RF05</b> | <b>Registro de Vendas</b>           | O sistema deve permitir registrar cada venda de doces. Ele deve calcular automaticamente o valor total da venda e, após a venda, diminuir a quantidade dos doces vendidos no estoque.                                                                                    |
| <b>RF06</b> | <b>Autenticação de Usuários</b>     | Para usar o sistema, os funcionários e o dono da doceria precisarão                                                                                                                                                                                                      |

|             |                             |                                                                                                                                                                                                                                       |
|-------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                             | fazer login com um nome de usuário e senha. O sistema vai gerar um “token” (uma espécie de chave temporária) para manter a sessão segura.                                                                                             |
| <b>RF07</b> | <b>Relatórios de Vendas</b> | O sistema deve ser capaz de gerar relatórios que mostrem as vendas em um determinado período (ex: vendas do mês passado), as vendas para um cliente específico e também gráficos que mostrem o desempenho das vendas ao longo do ano. |

### 3. Requisitos Não Funcionais (RNF)

Os requisitos não funcionais descrevem como o sistema deve funcionar, ou seja, suas qualidades e restrições. Eles não são sobre o que o sistema faz, mas como ele faz:

| ID           | Requisito         | Descrição                                                                                                                                                                                                                                                                          |
|--------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF01</b> | <b>Segurança</b>  | As senhas dos usuários devem ser guardadas de forma criptografada (codificadas para ninguém conseguir ler) usando uma técnica como o BCrypt. As partes importantes do sistema (como o registro de vendas) só poderão ser acessadas por quem tiver um token de autenticação válido. |
| <b>RNF02</b> | <b>Desempenho</b> | A API do sistema deve responder rapidamente às solicitações, entregando as                                                                                                                                                                                                         |

|       |                       |                                                                                                                                                                               |
|-------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                       | informações em formato JSON (um formato de dados fácil de entender por programas).                                                                                            |
| RNF03 | Manutenibilidade      | O sistema deve ser construído de forma organizada, em camadas, para que seja fácil de entender, fazer mudanças ou adicionar novas funcionalidades no futuro.                  |
| RNF04 | Usabilidade           | A interface do sistema (seja desktop ou web) deve ser simples e fácil de usar, para que os funcionários da doceria aprendam a utilizá-la rapidamente.                         |
| RNF05 | Integridade dos Dados | O sistema deve garantir que os dados estejam sempre corretos. Por exemplo, não deve ser possível apagar um cliente que já fez compras, para não perder o histórico de vendas. |
| RNF06 | Disponibilidade       | O sistema deve estar disponível para uso durante o horário de funcionamento da doceria, e deve ser capaz de lidar com várias vendas ao mesmo tempo sem perder informações.    |

---

#### 4. Arquitetura Proposta: Arquitetura em Camadas

Para construir o SGC da doceria, vamos usar uma **Arquitetura em Camadas**. Pense nisso como um bolo de várias camadas, onde cada camada tem uma função específica e bem definida. Isso ajuda a organizar o código e facilita a manutenção e o crescimento do sistema.

As camadas que vamos utilizar são:

1. **Camada de Apresentação (Frontend):** Esta é a camada que o usuário vê e interage. Pode ser uma tela de computador (se for Swing) ou uma página da web. Ela é responsável por mostrar as informações e coletar os dados que o usuário digita. Ela “conversa” com a próxima camada (Controller) para pedir e enviar informações.
2. **Camada de Controller (API REST):** Esta camada funciona como um “gerente de tráfego”. Ela recebe as requisições da Camada de Apresentação (ex: “quero cadastrar um novo doce”) e decide para qual parte do sistema essa requisição deve ir. Ela também é responsável por enviar as respostas de volta para a Camada de Apresentação, sempre em formato JSON.
3. **Camada de Serviço (Service/Lógica de Negócio):** Aqui é onde a “inteligência” do sistema mora. É nesta camada que as regras de negócio da doceria são aplicadas (ex: “verificar se o estoque é suficiente antes de vender”). Ela coordena as operações e garante que tudo seja feito corretamente, usando as informações da Camada de Domínio e pedindo para a Camada de Persistência guardar ou buscar dados.
4. **Camada de Domínio (Domain/Model):** Esta camada é o “coração” do sistema, onde definimos as entidades principais da doceria, como Cliente, Doce (Produto), Venda e Usuário. Aqui também ficam as regras mais básicas e intrínsecas a essas entidades (ex: “um doce não pode ter preço negativo”).
5. **Camada de Persistência (Repository/DAO):** Esta camada é responsável por “conversar” diretamente com o banco de dados. Ela sabe como guardar, buscar, atualizar e apagar as informações das entidades (clientes, doces, vendas) no MySQL. Usaremos o **Spring Data JPA** para facilitar essa comunicação, sem precisar escrever muito código SQL manualmente.

**Tecnologias que serão usadas:** Para construir o Backend, utilizaremos **Java 21** e o framework **Spring Boot 3**. Para guardar os dados, o banco de dados **MySQL**. Para gerenciar as dependências do projeto, o **Maven**. E para garantir a segurança da API, usaremos **JWT** para autenticação.

---

## 5. Padrões de Projeto Escolhidos

Para que o nosso sistema seja bem feito, organizado e fácil de manter, vamos usar alguns “Padrões de Projeto”. Pense neles como “receitas” ou “soluções testadas” para problemas comuns no desenvolvimento de software. Isso nos ajuda a escrever um código mais limpo e eficiente.

### *Padrão Repository*

- **Onde será aplicado:** Principalmente na **Camada de Persistência**. Teremos classes como `ClienteRepository`, `ProdutoRepository`, etc.
- **Por que foi escolhido:** Este padrão nos ajuda a separar a lógica de como acessamos o banco de dados da lógica de negócio. Assim, a Camada de Serviço não precisa saber os

detalhes de como os dados são guardados; ela apenas “pede” para o Repository buscar ou guardar um cliente, por exemplo.

- **Benefícios:** Torna o código mais organizado, facilita a escrita de testes (podemos simular o banco de dados) e, se um dia precisarmos trocar o MySQL por outro banco, as mudanças serão concentradas apenas no Repository.

#### *Padrão DTO (Data Transfer Object)*

- **Onde será aplicado:** Na comunicação entre a **Camada de Controller** (API) e a **Camada de Serviço**. Por exemplo, quando o Frontend envia dados para cadastrar um cliente, ele enviará um `ClienteDTO`.
- **Por que foi escolhido:** O DTO é como um “pacote de dados” simplificado. Ele nos permite enviar apenas as informações necessárias entre as camadas, sem expor todos os detalhes internos das nossas entidades de domínio. Isso é importante para a segurança e para manter a API mais “limpa”.
- **Benefícios:** Ajuda a proteger as entidades do nosso domínio, reduz a quantidade de dados trafegados e diminui o acoplamento (a dependência) entre a API e o modelo interno do sistema.

#### *Padrão Singleton*

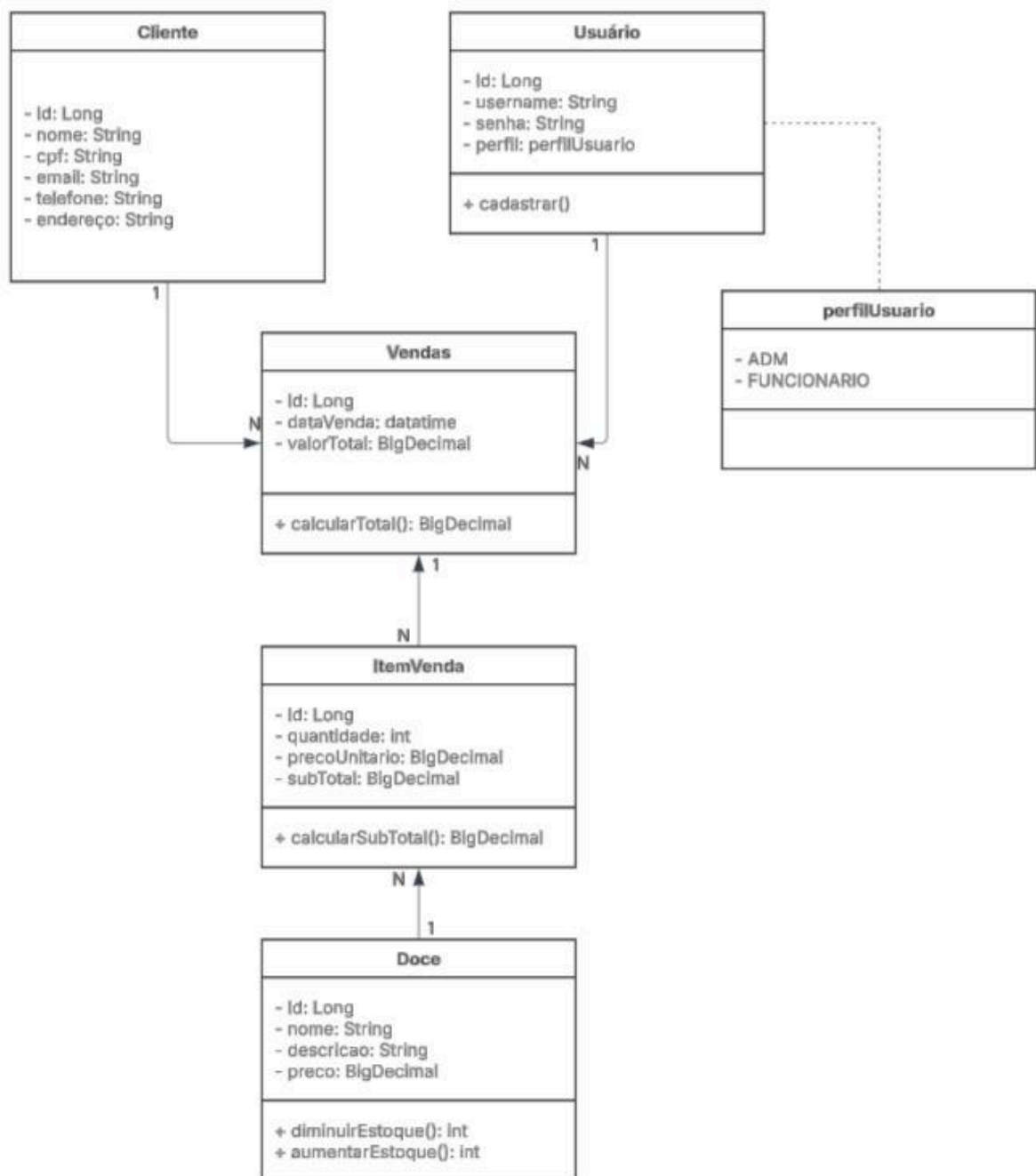
- **Onde será aplicado:** O Spring Framework, que usaremos, já gerencia automaticamente este padrão para muitos dos nossos componentes (chamados de “Beans”), como os serviços e as configurações de segurança.
- **Por que foi escolhido:** O Singleton garante que, para certas classes (como um serviço que gerencia clientes ou a configuração de segurança), exista apenas uma única instância (um único objeto) em toda a aplicação. Isso é útil para recursos que precisam ser compartilhados ou que são caros para criar várias vezes.
- **Benefícios:** Economiza memória, garante que todos os componentes estejam usando a mesma versão de um recurso e ajuda a manter a consistência do estado de componentes importantes.

---

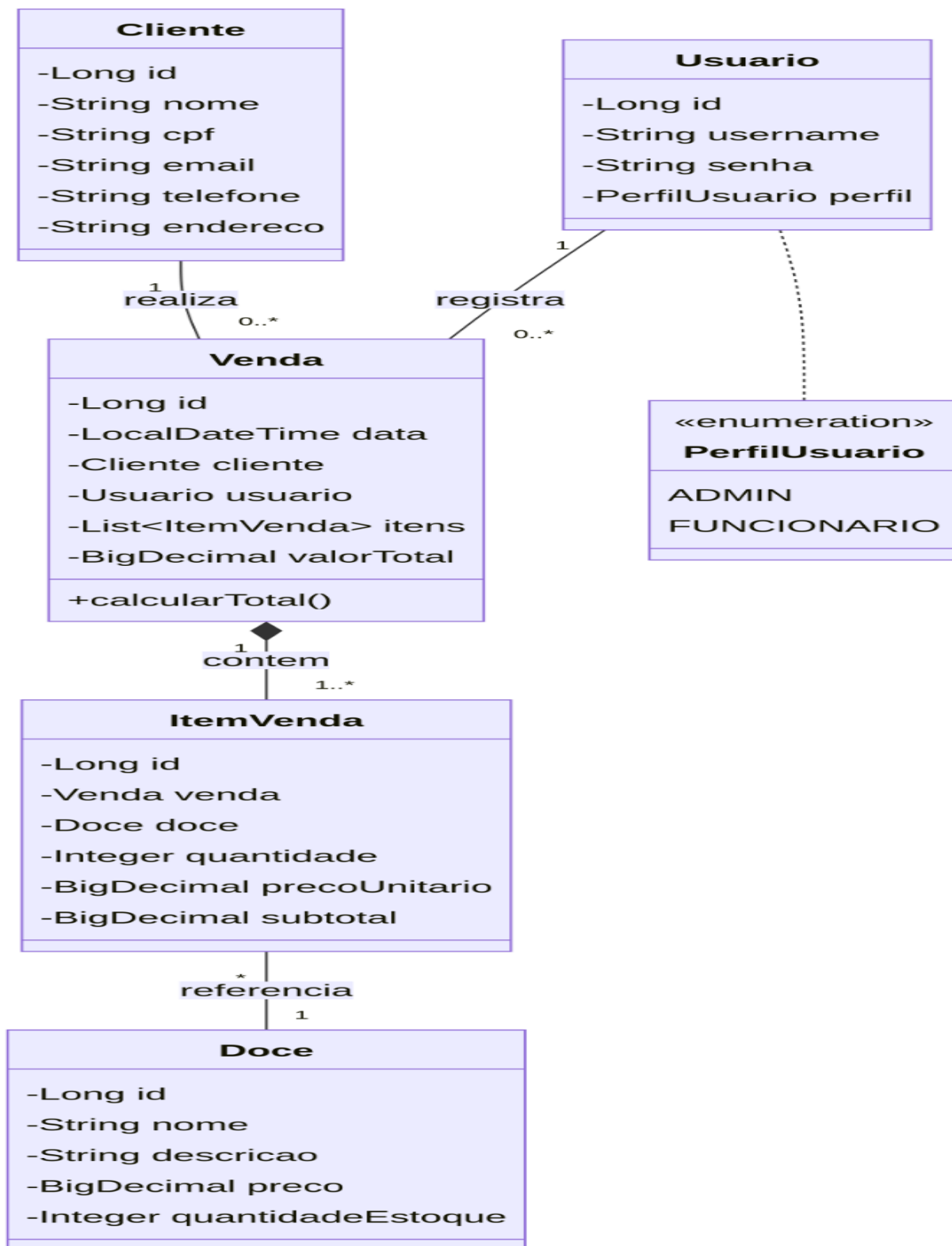
## **6. Diagramas e Scripts (Anexos Técnicos)**

Para complementar a documentação, serão fornecidos os seguintes anexos técnicos:

- Diagrama de Domínio:

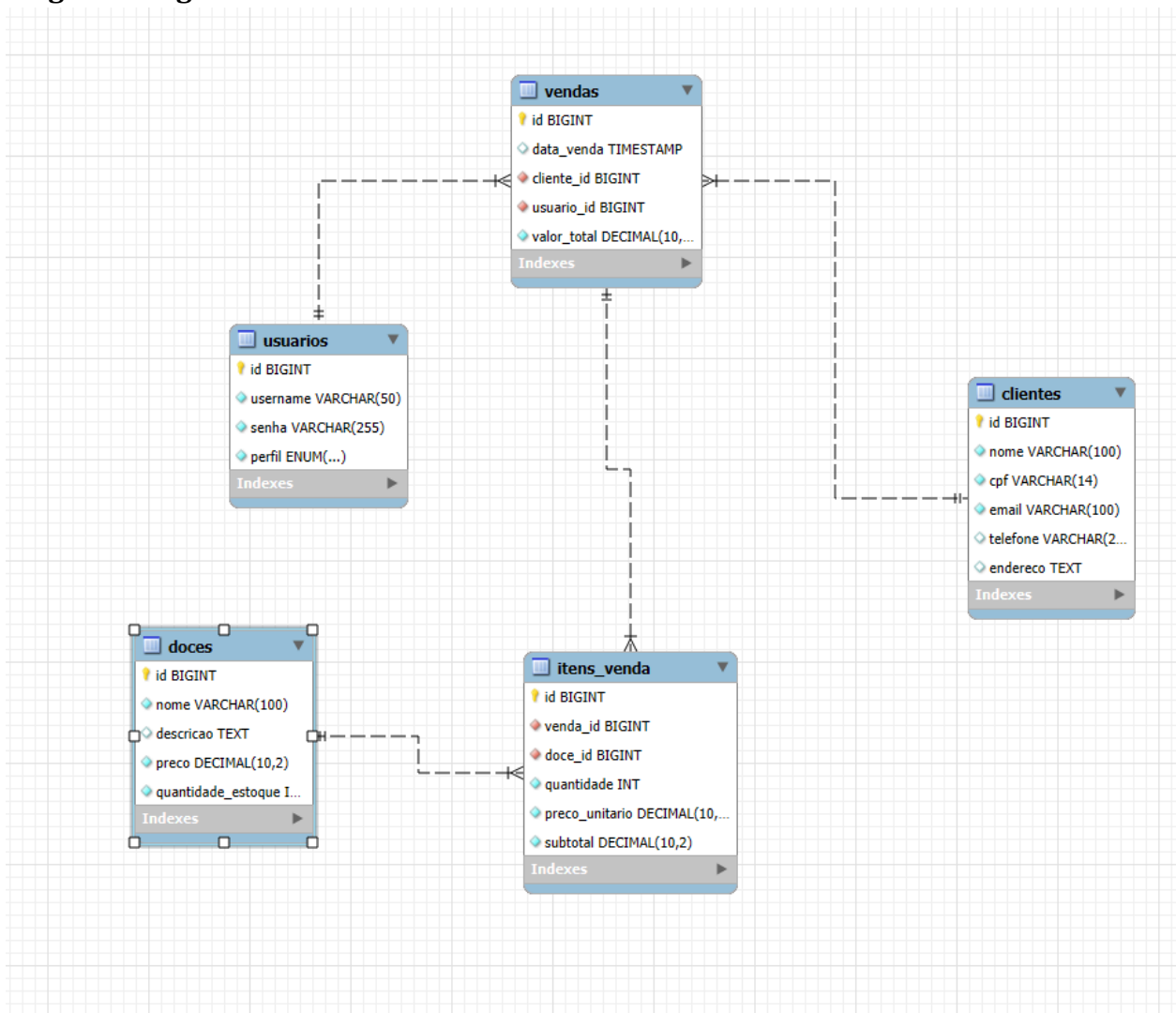


- Diagrama de Classes:





- **Diagrama Lógico do Banco de Dados:**



- **Script SQL:**

- -- Tabela de Usuários (para autenticação)
- CREATE TABLE IF NOT EXISTS usuarios (
- id BIGINT AUTO\_INCREMENT PRIMARY KEY,
- username VARCHAR(50) NOT NULL UNIQUE,
- senha VARCHAR(255) NOT NULL,
- perfil ENUM('ADMIN', 'FUNCIONARIO') NOT NULL
- );
- 
- -- Tabela de Clientes
- CREATE TABLE IF NOT EXISTS clientes (
- id BIGINT AUTO\_INCREMENT PRIMARY KEY,
- nome VARCHAR(100) NOT NULL,

- cpf VARCHAR(14) NOT NULL UNIQUE,
- email VARCHAR(100) NOT NULL,
- telefone VARCHAR(20),
- endereco TEXT
- );
- 
- -- Tabela de Doces
- CREATE TABLE IF NOT EXISTS doces (
- id BIGINT AUTO\_INCREMENT PRIMARY KEY,
- nome VARCHAR(100) NOT NULL,
- descricao TEXT,
- preco DECIMAL(10, 2) NOT NULL CHECK (preco >= 0),
- quantidade\_estoque INT NOT NULL DEFAULT 0 CHECK (quantidade\_estoque >= 0)
- );
- 
- -- Tabela de Vendas
- CREATE TABLE IF NOT EXISTS vendas (
- id BIGINT AUTO\_INCREMENT PRIMARY KEY,
- data\_venda TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,
- cliente\_id BIGINT NOT NULL,
- usuario\_id BIGINT NOT NULL,
- valor\_total DECIMAL(10, 2) NOT NULL DEFAULT 0,
- FOREIGN KEY (cliente\_id) REFERENCES clientes(id),
- FOREIGN KEY (usuario\_id) REFERENCES usuarios(id)
- );
- 
- -- Tabela de Itens de Venda
- CREATE TABLE IF NOT EXISTS itens\_venda (
- id BIGINT AUTO\_INCREMENT PRIMARY KEY,
- venda\_id BIGINT NOT NULL,
- doce\_id BIGINT NOT NULL,
- quantidade INT NOT NULL CHECK (quantidade > 0),
- preco\_unitario DECIMAL(10, 2) NOT NULL,
- subtotal DECIMAL(10, 2) NOT NULL,
- FOREIGN KEY (venda\_id) REFERENCES vendas(id),
- FOREIGN KEY (doce\_id) REFERENCES doces(id)
- );
- 
- -- Carga Inicial (Exemplo)
- -- A senha do admin está criptografada com BCrypt (exigência do trabalho)

- INSERT INTO usuarios (username, senha, perfil) VALUES ('admin', '\$2a\$10\$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd00DMxs.7uKCQqO', 'ADMIN');
- INSERT INTO doces (nome, descricao, preco, quantidade\_estoque) VALUES ('Bolo de Chocolate', 'Delicioso bolo de chocolate com cobertura', 45.00, 10);
- INSERT INTO doces (nome, descricao, preco, quantidade\_estoque) VALUES ('Brigadeiro Gourmet', 'Brigadeiro tradicional com granulado belga', 3.50, 100);

## Documento de Casos de Uso

### 1. Introdução

Este documento descreve os casos de uso para o sistema de vendas de doces, com base no diagrama de classes UML fornecido. O objetivo é detalhar as interações entre os atores e o sistema, definindo as funcionalidades essenciais.

### 2. Atores

Os atores identificados no sistema são:

6. **Cliente:** Entidade que realiza compras de doces.
7. **Funcionário:** Usuário do sistema responsável por registrar vendas e gerenciar o catálogo de doces.
8. **Administrador:** Usuário com privilégios elevados, responsável pela gestão de usuários e possivelmente relatórios do sistema.

### 3. Casos de Uso

A seguir, são detalhados os casos de uso do sistema:

#### 3.1. CU001: Realizar Venda

9. **Ator Principal:** Cliente
10. **Atores Secundários:** Funcionário
11. **Descrição:** Permite que um cliente compre um ou mais doces do catálogo disponível. O funcionário auxilia no processo de registro da venda.
12. **Pré-condições:**

1. O sistema deve estar operacional.
  2. Existem doces disponíveis em estoque.
13. **Pós-condições:**
1. Uma nova Venda é registrada no sistema.
  2. A quantidade em estoque dos doces vendidos é atualizada.
  3. O valor total da venda é calculado.
14. **Fluxo Principal:**
- O Cliente seleciona os doces desejados.
  - O Funcionário registra os itens selecionados no sistema.
  - O sistema verifica a disponibilidade dos doces e atualiza o subtotal de cada item.
  - O sistema calcula o valor total da venda.
  - O Funcionário finaliza a venda.
  - O sistema registra a venda e atualiza o estoque.

### 3.2. CU002: Registrar Venda

15. **Ator Principal:** Funcionário
16. **Descrição:** Permite que um funcionário registre uma nova venda no sistema, associando-a a um cliente e adicionando os itens de venda.
17. **Pré-condições:**
1. O funcionário está autenticado no sistema.
  2. Existe um cliente cadastrado ou o cliente é cadastrado no momento da venda.
18. **Pós-condições:**
1. Uma nova Venda é criada e associada a um Cliente e ao Usuário (Funcionário) que a registrou.
  2. Os itens de venda são adicionados à Venda.
  3. O valor total da Venda é calculado.
19. **Fluxo Principal:**
- O Funcionário inicia o processo de registro de uma nova venda.
  - O Funcionário seleciona ou cadastra o Cliente.
  - Para cada doce que o Cliente deseja comprar:
    - a. O Funcionário seleciona o Doce.
    - b. O Funcionário informa a quantidade desejada.
    - c. O sistema verifica a disponibilidade e adiciona o ItemVenda à Venda.
  - O Funcionário solicita o cálculo do valor total da Venda.
  - O sistema calcula o valorTotal da Venda e o subtotal de cada ItemVenda.
  - O Funcionário confirma o registro da Venda.
  - O sistema persiste a Venda e atualiza a quantidadeEstoque dos Doces.

### 3.3. CU003: Gerenciar Doces

20. **Ator Principal:** Funcionário

21. **Descrição:** Permite que um funcionário adicione novos doces ao catálogo, edite informações de doces existentes ou remova doces do catálogo.

22. **Pré-condições:**

1. O funcionário está autenticado no sistema.

23. **Pós-condições:**

1. O catálogo de doces é atualizado (doce adicionado, editado ou removido).

24. **Fluxo Principal (Adicionar Doce):**

- O Funcionário seleciona a opção para adicionar um novo doce.
- O Funcionário insere o nome, descrição, preço e quantidade em estoque do novo doce.
- O sistema valida as informações e adiciona o doce ao catálogo.

25. **Fluxo Principal (Editar Doce):**

- O Funcionário seleciona a opção para editar um doce existente.
- O Funcionário busca e seleciona o doce a ser editado.
- O Funcionário modifica as informações desejadas (nome, descrição, preço, quantidade em estoque).
- O sistema valida as informações e atualiza o doce no catálogo.

26. **Fluxo Principal (Remover Doce):**

- O Funcionário seleciona a opção para remover um doce.
- O Funcionário busca e seleciona o doce a ser removido.
- O sistema solicita confirmação.
- O Funcionário confirma a remoção.
- O sistema remove o doce do catálogo.

### 3.4. CU004: Gerenciar Usuários

27. **Ator Principal:** Administrador

28. **Descrição:** Permite que um administrador adicione, edite ou remova usuários do sistema, bem como gerencie seus perfis (ADMIN/FUNCIONARIO).

29. **Pré-condições:**

1. O administrador está autenticado no sistema.

30. **Pós-condições:**

1. As informações de usuários do sistema são atualizadas.

31. **Fluxo Principal (Adicionar Usuário):**

- O Administrador seleciona a opção para adicionar um novo usuário.
- O Administrador insere o username, senha e perfil do novo usuário.
- O sistema valida as informações e cria o novo usuário.

**32. Fluxo Principal (Editar Usuário):**

- O Administrador seleciona a opção para editar um usuário existente.
- O Administrador busca e seleciona o usuário a ser editado.
- O Administrador modifica as informações desejadas (username, senha, perfil).
- O sistema valida as informações e atualiza o usuário.

**33. Fluxo Principal (Remover Usuário):**

- O Administrador seleciona a opção para remover um usuário.
- O Administrador busca e seleciona o usuário a ser removido.
- O sistema solicita confirmação.
- O Administrador confirma a remoção.
- O sistema remove o usuário.

### **3.5. CU005: Visualizar Vendas**

**34. Ator Principal:** Funcionário

**35. Atores Secundários:** Administrador

**36. Descrição:** Permite que um funcionário ou administrador visualize o histórico de vendas realizadas no sistema.

**37. Pré-condições:**

1. O usuário (funcionário ou administrador) está autenticado no sistema.

**38. Pós-condições:**

1. O histórico de vendas é exibido.

**39. Fluxo Principal:**

- O usuário seleciona a opção para visualizar vendas.
  - O sistema recupera e exibe a lista de vendas, incluindo detalhes como data, cliente, usuário que registrou e valor total.
  - O usuário pode filtrar ou ordenar as vendas (opcional).
-